

RE Leçon 1b — C Avancé

1. Arithmétique de pointeurs

Un pointeur peut être incrémenté. Il avance en mémoire de la taille du type qu'il pointe.

```
#include <stdio.h>

int main() {
    int tableau[] = {10, 20, 30, 40, 50};
    int *p = tableau; // p pointe vers le premier élément

    printf("p[0] = %d\n", *p); // 10
    p++; // avance de 4 octets (taille int)
    printf("p[1] = %d\n", *p); // 20
    p++;
    printf("p[2] = %d\n", *p); // 30

    // Parcourir un tableau avec un pointeur
    p = tableau;
    int i;
    for (i = 0; i < 5; i++) {
        printf("tableau[%d] = %d (adresse: %p)\n", i, *(p+i), (p+i));
    }

    return 0;
}
```

Ce que le RE retient : en assembleur, `mov eax, [rbx+4]` = accès à l'élément suivant d'un tableau d'entiers. Reconnaître ce pattern permet d'identifier les tableaux dans un binaire.

2. malloc / free — mémoire heap

`malloc` alloue de la mémoire dynamiquement sur le **heap**. Contrairement à la stack, elle persiste jusqu'à ce qu'on la libère.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // Allouer 50 octets sur le heap
    char *buffer = (char *)malloc(50);

    if (buffer == NULL) {
        printf("Erreur d'allocation\n");
        return 1;
    }
}
```

```
}

strcpy(buffer, "données en heap");
printf("buffer : %s\n", buffer);
printf("adresse heap : %p\n", buffer);

// Toujours libérer la mémoire
free(buffer);
buffer = NULL;

return 0;
}
```

Ce que le RE retient : `call malloc` dans un désassemblé = allocation dynamique. L'argument dans `rdi` = taille demandée. Le retour dans `rax` = adresse du bloc alloué. Pattern très fréquent dans les malwares.

3. Pointeurs de fonctions

Une fonction peut être stockée dans un pointeur et appelée via ce pointeur.

```
#include <stdio.h>

int additionner(int a, int b) { return a + b; }
int soustraire(int a, int b) { return a - b; }
int multiplier(int a, int b) { return a * b; }

int main() {
    // Déclarer un pointeur de fonction
    int (*operation)(int, int);

    operation = additionner;
    printf("add : %d\n", operation(10, 5));    // 15

    operation = soustraire;
    printf("sub : %d\n", operation(10, 5));    // 5

    operation = multiplier;
    printf("mul : %d\n", operation(10, 5));    // 50

    // Tableau de pointeurs de fonctions
    int (*ops[3])(int, int) = {additionner, soustraire, multiplier};
    int i;
    for (i = 0; i < 3; i++) {
        printf("ops[%d](6,2) = %d\n", i, ops[i](6, 2));
    }

    return 0;
}
```

Ce que le RE retient : `call rax` ou `call [rbx+0x10]` en assembleur = appel via pointeur de fonction.
Pattern classique dans les malwares pour rendre le code difficile à analyser statiquement.

4. Opérations bitwise — les plus importantes en RE

Les opérations bit à bit sont omniprésentes : chiffrement XOR, masques, flags, encodage.

```
#include <stdio.h>

int main() {
    unsigned char a = 0b10110100; // 180
    unsigned char b = 0b11001010; // 202

    printf("a          = %d (0x%02X)\n", a, a);
    printf("b          = %d (0x%02X)\n", b, b);
    printf("a AND b       = %d (0x%02X)\n", a & b, a & b);
    printf("a OR  b        = %d (0x%02X)\n", a | b, a | b);
    printf("a XOR b        = %d (0x%02X)\n", a ^ b, a ^ b);
    printf("NOT a         = %d (0x%02X)\n", ~a & 0xFF, ~a & 0xFF);
    printf("a << 2        = %d (0x%02X)\n", a << 2, a << 2); // shift gauche
    printf("a >> 2        = %d (0x%02X)\n", a >> 2, a >> 2); // shift droite

    // XOR encryption – technique classique en malware
    char message[] = "SECRET";
    char cle = 0x42;
    int i;

    printf("\nMessage original : %s\n", message);

    // Chiffrer
    for (i = 0; i < 6; i++)
        message[i] ^= cle;
    printf("Chiffré (XOR 0x42) : ");
    for (i = 0; i < 6; i++)
        printf("%02X ", (unsigned char)message[i]);
    printf("\n");

    // Déchiffrer (XOR deux fois = retour à l'original)
    for (i = 0; i < 6; i++)
        message[i] ^= cle;
    printf("Déchiffré : %s\n", message);

    return 0;
}
```

Ce que le RE retient :

- `xor eax, eax` = mettre `eax` à 0 (pattern très courant)
- `xor` répété avec la même clé = chiffrement simple (malware basique)
- `and eax, 0xFF` = isoler le byte de poids faible

- `shl / shr` = multiplier/diviser par une puissance de 2

5. Unions

Une union permet à plusieurs variables de **partager le même espace mémoire**.

```
#include <stdio.h>

union Data {
    int    entier;        // 4 octets
    float  flottant;      // 4 octets
    char   bytes[4];      // 4 octets
};

int main() {
    union Data d;

    d.entier = 0x41424344; // 'DCBA' en ASCII

    printf("entier   : %d\n", d.entier);
    printf("float    : %f\n", d.flottant);
    printf("bytes    : %c %c %c %c\n",
           d.bytes[0], d.bytes[1], d.bytes[2], d.bytes[3]);
    printf("hex      : %02X %02X %02X %02X\n",
           (unsigned char)d.bytes[0], (unsigned char)d.bytes[1],
           (unsigned char)d.bytes[2], (unsigned char)d.bytes[3]);

    return 0;
}
```

Ce que le RE retient : les unions montrent que la même zone mémoire peut être interprétée de plusieurs façons selon le type. En RE, on voit souvent du code qui reinterprète des bytes en int ou float — c'est le même mécanisme.

6. Cast de types

Forcer une variable à changer de type.

```
#include <stdio.h>

int main() {
    // Cast implicite
    int x = 65;
    char c = (char)x;        // 65 = 'A' en ASCII
    printf("int %d = char '%c'\n", x, c);

    // Cast de pointeurs – technique RE classique
    int nombre = 0xDEADBEEF;
```

```
char *octets = (char *)&nombre;
int i;

printf("0xDEADBEEF en bytes : ");
for (i = 0; i < 4; i++) {
    printf("%02X ", (unsigned char)octets[i]);
}
printf("\n");

// Pointeur void – générique
void *ptr = &nombre;
printf("via void* : %d\n", *(int *)ptr);

return 0;
}
```

Ce que le RE retient : le cast de pointeurs permet de lire n'importe quelle zone mémoire comme un type arbitraire. Pattern fréquent dans les shellcodes et malwares pour manipuler des données brutes.

Récapitulatif RE

Notion C	Pattern assembleur associé
p++ sur int*	add rbx, 4
malloc(n)	mov rdi, n + call malloc
call via pointeur	call rax ou call [rbx+offset]
a XOR b	xor eax, ebx
xor x, x	mise à zéro d'un registre
Cast (char*)&int	accès byte par byte à un entier